



PDF Download
3122863.pdf
28 February 2026
Total Citations: 10
Total Downloads: 266

 Latest updates: <https://dl.acm.org/doi/10.1145/3122863>

RESEARCH-ARTICLE

Cluster-Based Quality-Aware Adaptive Data Compression for Streaming Data

ASEEL BASHEER, University of Baghdad, Baghdad, Iraq

KEWEI SHA, University of Houston, Houston, TX, United States

Open Access Support provided by:

University of Baghdad

University of Houston

Published: 21 September 2017

Accepted: 01 July 2017

Revised: 01 April 2017

Received: 01 November 2016

[Citation in BibTeX format](#)

Cluster-Based Quality-Aware Adaptive Data Compression for Streaming Data

ASEEL BASHEER, Department of Computer Science, College of Science for Women University of Baghdad, Baghdad, Iraq

KEWEI SHA, Department of Computer Science & Cyber Security Institute University of Houston

Wireless sensor networks (WSNs) are widely applied in data collection applications. Energy efficiency is one of the most important design goals of WSNs. In this article, we examine the tradeoffs between the energy efficiency and the data quality. First, four attributes used to evaluate data quality are formally defined. Then, we propose a novel data compression algorithm, Quality-Aware Adaptive data Compression (QAAC), to reduce the amount of data communication to save energy. QAAC utilizes an adaptive clustering algorithm to build clusters from dataset; then a code for each cluster is generated and stored in a Huffman encoding tree. The encoding algorithm encodes the original dataset based on the Huffman encoding tree. An improvement algorithm is also designed to reduce the information loss when data are compressed. After the encoded data, the Huffman encoding tree and parameters used in the improvement algorithm have been received at the sink, a decompression algorithm is used to retrieve the approximation of the original dataset. The performance evaluation shows that QAAC is efficient and achieves a much higher compression ratio than lossy and lossless compression algorithms, while it has much smaller information loss than lossy compression algorithms.

CCS Concepts: • **Information systems** → **Information systems applications**; **Spatial-temporal systems**; **Sensor networks**;

Additional Key Words and Phrases: Wireless sensor networks, data quality, adaptive, clustering algorithm, energy efficiency, data compression

ACM Reference format:

Aseel Basheer and Kewei Sha. 2017. Cluster-Based Quality-Aware Adaptive Data Compression for Streaming Data. *J. Data and Information Quality* 9, 1, Article 2 (September 2017), 33 pages.

<https://doi.org/10.1145/3122863>

1 INTRODUCTION

Wireless sensor networks (WSNs) are suitable for large-scale data gathering, and they have become increasingly important for enabling continuous monitoring in many fields especially because Internet of Things (IoT) (Chi et al. 2014; Lin et al. 2017) is becoming more and more popular. Many applications using WSNs in areas such as environmental monitoring, industrial monitoring, health and wellness monitoring, inventory location monitoring, surveillance, power monitoring, object tracking, precision agriculture, and disaster management (Kolo et al. 2012). In these

Authors' addresses: A. Basheer, Department of Computer Science, College of Science for Women University of Baghdad, Baghdad, Iraq; email: asbasheer@cs.w.uobaghdad.edu.iq; K. Sha (corresponding author), Department of Computer Science & Cyber Security Institute University of Houston, Clear Lake, Houston, TX 77058; email: sha@uhcl.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2017 ACM 1936-1955/2017/09-ART2 \$15.00

<https://doi.org/10.1145/3122863>

systems, climate-related data such as temperature, humidity, wind speed, and so on, are collected using wireless sensors like motes; then the data are transferred to a sink, which is usually a much more powerful device and works as the gateway connecting the sensor networks and the Internet (Akyildiz et al. 2002). Most sensors used in WSNs are resource-constrained devices. For example, they have limited communication bandwidth and power supply, small storage spaces, and not-so-powerful processors (Kwon and Cioffi 2013).

Among all the above limitations, battery-backed power is the most critical one to the lifetime of WSNs. Moreover, it is difficult or very costly to replace the battery of the sensors (Kwon and Cioffi 2013). It has also been shown that communication consumes most energy in WSNs (Razzaque et al. 2013); that is, approximately 80% of power consumed is used for data transmission, so data transmission should be minimized to save energy and maximize network lifetime. Many techniques such as data aggregation (Fasolo et al. 2007; Yu et al. 2004; Zhang et al. 2013) and data compression (Aboelela 2014) have been proposed to save energy by reducing the amount of the data that is transmitted from the sensors to the sink. Although data aggregation can significantly reduce the amount of data by using aggregation functions, it makes the value of the original data invisible so it may also degrade the quality of the received data at the sink. On the other hand, data compression can also be used to reduce the amount of the data while consider data quality. Comparing two classes of data compression algorithms, lossy algorithms usually have a higher compression rate but introduce information loss, while lossless ones have no impact on data quality but less applicable in WSNs because of constrained resources at sensors (Sadler and Martonosi 2006). Moreover, we also observe that a lot of sensor applications can tolerate a certain level of information loss (Srisooksai et al. 2012), so lossy data compression algorithms are more preferred in WSNs if the information loss by the algorithm can be bounded to satisfy the minimum data quality requirements of the application.

This work proposes a novel Quality-Aware Adaptive data Compression (QAAC) algorithm that aims to reduce the amount of data in transmission and guarantee the quality of the transmitted data. First, formal models evaluating data quality are defined. Then, an efficient clustering algorithm builds clusters from a dataset and each cluster is assigned a cluster code using Huffman encoding method. Based on the cluster code, the original collected data are encoded using an encoding algorithm with an improvement approach that further limits the information loss. The data quality control is embedded in the design of both the clustering algorithm and the encoding algorithm. After the encoded data and the Huffman tree have been received at the sink, a decompression algorithm is used to retrieve the approximation of the original data. A comprehensive performance evaluation shows that QAAC outperforms existing lossy and lossless compression algorithms in terms of compression ratio, that is, the lowest compression ratio of QAAC is 73% while it is 0.2% for some other lossy compression algorithms and zero for lossless ones for a similar level of data quality. Moreover, our work can achieve a very high compression ratio of 98% in the best case. Comparing with other lossy compression algorithms, QAAC also achieves smaller information loss, that is, it delivers a set of data mostly close to the original collected data. The data quality is further improved when the improvement approach is applied in the encoding algorithm.

The main contribution of this article is threefold. First, we propose the concept of quality-aware data compression and define formal models to evaluate data quality. Second, we propose a novel cluster-based data compression scheme with an optimized strategy to limit the information loss when the data are compressed. Third, we conduct comprehensive performance evaluation over eight real-world datasets by comparing the proposed algorithm with seven other lossless and lossy data compression algorithms. The experimental results show that our work outperforms all compared lossy algorithms in terms of compression ratio and information loss.

The rest of the article is organized as follows. In Section 2, we present the quality-aware data compression problem and the motivation of our work. Formal data-quality models are defined in Section 3. Section 4 describes the proposed QAAC algorithm in detail, which is followed by a comprehensive performance evaluation in Section 5. Related work is listed in Section 6, and, finally, we conclude the article and identify future work in Section 7.

2 MOTIVATION

A WSN is a network formed by a large number of resource constrained sensor nodes where each node is equipped with sensors to collect data such as temperature, light, pressure, and so on. A typical example of sensor node is Crossbow Mica2, MPR400 (Board and Interface 2003), which is an 8-bit RISC microcontroller with 128Kbytes of programmable Flash memory, 4Kbytes EEPROM, 4Kbytes internal SRAM, and powered by 2 AA batteries, which place stringent energy constraints on the design of all WSN operations.

Prolonging the lifetime of a WSN is important for both commercial and tactical applications. Non-rechargeable batteries places stringent energy constraints on the design of all WSN operations. Since the early 2010s, a lot of effort has been made to reduce energy consumption and extend the lifetime of WSNs (Razzaque et al. 2013; Wang et al. 2014). Some of them aim at minimizing the energy spent in the input/output operations at data transmission levels (Razzaque et al. 2013), while others target at saving energy by means of the formulation of optimal topology and the design of energy efficiency routing schemes (Wang et al. 2014). Energy is consumed in sensing, communication, data processing, and data storage, but the predominant amount of energy is consumed in communication and EEPROM access (Shnayder et al. 2004). Thus, to maximize the lifetime of WSNs and save communication bandwidth, it is essential for each sensor to locally process the collected data thus to reduce the amount of transmitted data. Sending fewer data may result in information loss. Therefore, it is necessary to design mechanisms not only to reduce the amount of transmitted data but also to guarantee the level of data quality required by the application. One major technique used to reduce the amount of data is data aggregation (Fasolo et al. 2007), that is, after sensor data has been collected, aggregation functions are applied to the data and only the aggregated result will be sent to the sink. This design can significantly reduce the amount of the energy used in the communication, but it also brings new problems. The original values cannot be recovered after having been processed by means of the aggregation function, although the original data may be needed by some applications. In addition, we may lose precision of the original data (Fasolo et al. 2007). Finally, aggregation also adds difficulty for faulty data detection (Sha and Zeadally 2015). Other work (Rajagopalan and Varshney 2006) provides a comprehensive survey about data aggregation techniques in WSNs and their limitations. Other mechanisms are needed to reduce the amount of transmitted data with consideration of data quality.

Data compression is the other popular approach to reduce the amount of the transferred data. There are many data compression algorithms existing in the literature (Sayood 2012). Most of them are either general data compression algorithms or designed specifically for one certain type of data such as image data. Most conventional compression algorithms are not directly applicable to WSNs (Razzaque et al. 2013) in that they usually have a different objective that does not focus on energy conservation. In addition, WSN applications may tolerate a different level of information loss compared with other applications, but most conventional compression algorithms do not adapt to application's information loss requirements.

In summary, in-network processing such as compression and aggregation, which processes data locally at sensors before it is sent to the sink, is a popular technique to optimize the traffic and reduce the load to the network, so a lot of energy can be saved, but novel in-network processing mechanisms that consider both data reduction and data quality are needed. This work aims to

fill the above gap. We believe that it is extremely important to guarantee the quality of the data that will be received at the sink and used in many applications to make critical decisions. We also observe that different applications have specific requirements on data quality. For example, some applications such as tornado tracking require very accurate, precise, and complete data, while others such as city temperature monitoring may have a much lower data-quality requirement. Therefore, in this work, we propose to design a quality-aware data compression algorithm, which is adaptive to fulfil specific data-quality requirements of applications.

3 DATA QUALITY DEFINITION

Data quality is a very important concept in WSNs, because the success of WSNs applications depends on the quality of data collected (Sha and Shi 2008). The real concern with data quality is to ensure that data is accurate enough to make appropriate and reliable decisions based on the collected data (Blake and Mangiameli 2011). Quality of data can be measured using different attributes, including accuracy, precision, completeness, and consistency (Blake and Mangiameli 2011; Hazen et al. 2014).

In general, **Accuracy** is the degree of closeness of measurements to the true value (Blake and Mangiameli 2011). Cappiello et al. (2009) treats accuracy as the error expressed by the difference between the mean of the measurements and a reference value, but it could be misleading, because comparing the mean value only may not satisfy the application's accuracy requirements. We measure the accuracy of a sensed data as the difference between the value of each piece of retrieved data at the sink and the actual value sensed at the sensor. Two popular metrics, Root Mean Square Error (Guestrin et al. 2004) and Mean Absolute Error (Witten and Frank 2005), are widely used to measure accuracy. They are defined as follows.

Assume $X_1, \dots, X_n$ as a series of n values received at the sink, which is meant to represent n corresponding true sensor readings, $N_1, \dots, N_n$, and error can be defined as the difference between them that is $e_i = X_i - N_i$.

— **Root Mean Square error** (RMSE) has been used as a standard statistical metric of accuracy to measure information loss in meteorology, air quality, and climate research studies (Savage et al. 2013). RMSE is defined as

$$e_{RMSE} = \sqrt{e_{MSE}} = \sqrt{\frac{1}{n} \sum_{i=1}^n e_i^2}. \quad (1)$$

— **Mean Absolute Error** (MAE) is another metric to evaluate data accuracy and is a common measure of forecast error in time-series analysis (Jerez et al. 2013). It measures the average unsigned error in the received data sequence. The MAE is defined as

$$e_{MAE} = \frac{1}{n} \sum_{i=1}^n |e_i|. \quad (2)$$

Accuracy is related to *precision*, that is, the degree to which the original and retrieved datasets show the same or similar results (Commission et al. 2007). Precision is often characterized in terms of the standard deviation of the measured values. Standard deviation is measured based on the following equation:

$$\frac{\sum_{i=1}^n (X_i - \mu)^2}{n}, \quad (3)$$

where μ is the mean of the data.

For example, if the retrieved dataset is not accurate but precise, then it means that values in it are not similar to that in the original dataset, but they are characterized by a close standard deviation (Cappiello et al. 2009). The smaller difference in the standard deviation between the retrieved dataset and original dataset means that the two datasets have a similar distribution, and therefore more precision is achieved.

Completeness is a property of a set of sensor readings that captures the minimally required amount of information needed (Wang et al. 1995). Completeness does not mean lossless. In the lossless data compression, all data points can be recovered exactly the same as the set of original data points after decompression, while completeness can still be maintained although some data points are different from the original data points. In other words, if the sink has received enough data samples to construct a similar set of data, which contains very close amount of information compared with the original data set, then the set of samples is complete. In this article, the completeness of data is measured based on entropy (Shannon 2001) that is calculated from the probabilities of occurrence of unique symbols as in the formula below:

$$H = - \sum_{i=1}^n p_i \log p_i, \quad (4)$$

where p_i is the probability of occurrence of symbol X_i in the set and n is the total number of unique symbols.

Based on the findings in Balakrishnan and Touba (2007), the entropy determines the minimum average number of bits required for each codeword. Since our work focuses on minimizing the number of bits that is needed to assure the quality of data, a smaller entropy value may imply a higher compression ratio. In addition to that, authors in Balakrishnan and Touba (2007) also use entropy to determine the maximum of lossless compression ratio that can be reached. More specifically, they use the following formula:

$$\text{Max compression} = \frac{\text{symbol_length} - \text{entropy}}{\text{symbol_length}} * 100\%. \quad (5)$$

Consistency: It is a sub-aspect of accuracy. If a single piece or a set of data is compliant with a user-defined model, then it is consistent (Lang and Wilkerson 2008; Sha and Shi 2008). A user-defined model makes an assumption about the retrieved data. For instance, a model may require that the difference between the original and the retrieved temperature reading should not exceed a threshold (θ) like 0.5°C . Absolute error can be used to measure the consistency of the data. We calculate absolute error as the absolute difference between each retrieved datum by the base station, N_i , and its actual sensed value from the environment, X_i . The data are consistent if it satisfies the following condition:

$$|X_i - N_i| \leq \theta. \quad (6)$$

In conclusion, data quality needs to be considered when lossy compression is applied to reduce the amount of the transferred data. Based on the above definitions, we expect that the compressed data received at the sink preserves the properties of accuracy, precision, completeness, and consistency.

4 QAAC: QUALITY-AWARE ADAPTIVE DATA COMPRESSION

In this section, we present the detailed design of our proposal: QAAC. We assume that a WSN consists of a set of sensor nodes and one or more sink nodes. Each sensor node is equipped with m sensors, $S_1, S_2, \dots, S_m$, performing readings $X_1, \dots, X_n$ from sensor S_i . Our approach makes no

Table 1. All Key Notations in This Paper

Symbol	meaning
α	a parameter used in the λ calculation, $\alpha \in (0,1)$
λ	the percentage of θ that is $\lambda = \alpha\theta$
μ_i	mean
θ	threshold (maximum information loss)
ave	average distance between the cluster center and the points far away from the cluster center
c_i	cluster center
c_{new}	new cluster center
$decodedData_i$	decoded data at the sink
$dist_i$	distance between a data point and its cluster center
e_i	difference between the original value and retrieved value of a data point
$encoded$	encoded data at sensor node
H	entropy of a dataset
I	number of iterations
k	number of clusters
K	number of times a dictionary is stored
k_{new}	number of clusters after a round of stabilization
k_{old}	number of clusters before a round of stabilization
l	size of dictionary
m	dimensions of a dataset
max_c	maximum reading in a cluster
min_c	minimum reading in a cluster
n	size of dataset
N	length of each data point
$orig_i$	original value of a data point
p_i	probability of the occurrence of a symbol in a dataset
r_i	radius of a cluster
r_{new}	radius of a new cluster
$receivedData_i$	data point received at the sink
$RMSE_{new}$	RMSE value after a round of stabilization
$RMSE_{old}$	RMSE value before a round of stabilization
S_i	a sensor node
t_i	an attribute of a data point
X_i	a sensor reading
X_{max}	maximum reading in a dataset
X_{min}	minimum reading in a dataset

assumption of the routing protocol or the network topology. Table 1 lists all key notations used in this article.

QAAC makes a tradeoff between data quality and the amount of transmitted data, that is, we expect to deliver a minimum amount of data while satisfying data-quality requirements of the application. To achieve the above goals, QAAC needs to collect a set of data and compresses it before it is sent to the sink. This mechanism is especially useful in WSN applications where data

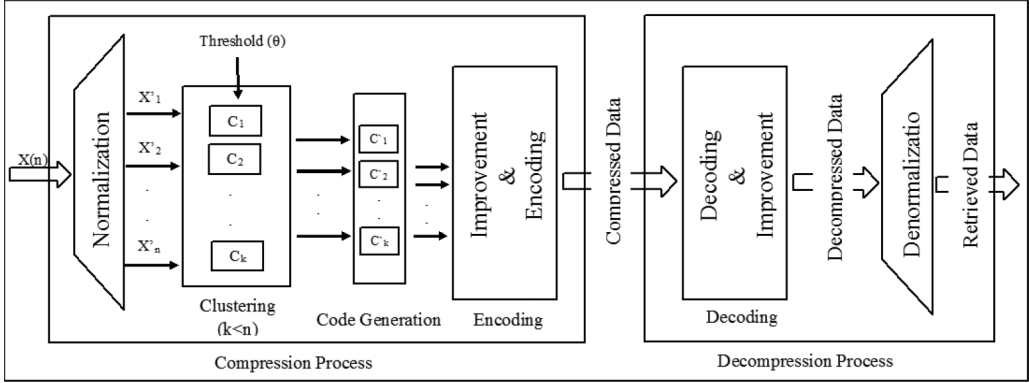


Fig. 1. An overview of QAAC.

are collected periodically by data mules (Al-Karaki and Kamal 2004; Whitehead 2016). It can also be applied in many other delay-tolerant WSN applications (Malavenda et al. 2016) and WSN applications that generate a huge amount of data in a short period of time. In addition, a tradeoff can be made between the compression ratio and the real-time requirement so the compression can be performed based on a small set of data. An online data compression algorithm will be our future work. QAAC can be formally defined as

$$QAAC(\langle X_1, \dots, X_n \rangle, \theta) \rightarrow (C_1, \dots, C_k), \quad (7)$$

where θ is the threshold specifies the application's maximum information loss toleration. The difference between the original data and retrieved one should not exceed that threshold, and $X_1, \dots, X_n$ are the original sensor readings while the $C_1, \dots, C_k$ are the compressed data. An overview of the design of QAAC is depicted in Figure 1. QAAC consists of two major steps, data compression before data transmission and data decompression after data are received. Next, we illustrate the details of each step. In the phase of data compression, the data will be normalized into scale $[0,1]$ and θ will be normalized based on the dataset itself. After that, the data will be partitioned into clusters based on the closeness among the data. Following that, a Huffman encoding tree is generated to store the codes for clusters based on the size of clusters. Then, the original dataset will be encoded using an encoding algorithm, and an improvement approach is also adopted to further reduce the information loss. The encoded data, a parameter in the improvement approach, and the Huffman encoding tree will be sent to the sink to do the decompression. Decompression is performed at the sink to retrieve the approximation of original data based on the received data. First, the data will be decoded using the decoding algorithm with improvement applied when necessary. Then, the data will be denormalized. Next, we illustrate the detail of each step.

4.1 Normalization

Normalization before clustering is needed, because distance metrics, such as Euclidian distance used in a k -means clustering algorithm, may be sensitive to differences in the magnitude or scales of the attributes. For example, because of the differences in scale of attributes, one attribute might overpower others in distance calculation. We adopt Min-Max normalization (Visalakshi and Thangavel 2009) that performs a linear transformation mapping the original data to an interval of $[0,1]$. Please note that as an application-specific parameter when the value of the adaptive parameter, θ , is chosen by the WSN application scientists, it is not a normalized value; therefore in the algorithm, the value of the θ needs to be normalized accordingly.

4.2 Adaptive Clustering

Our design of compression is to use the same short code to represent many sensor readings that have close value. Therefore, an adaptive clustering algorithm is designed to find sensor readings that are close to each other. We develop the adaptive clustering algorithm by modifying the existing k -means algorithm (MacQueen et al. 1967) because of the following considerations. First, to bound the information loss to preserve the quality of data, a spherical shape clustering algorithm such as k -means is a preferred choice. In this way, the applications's data quality requirements can be converted to the radius of the sphere. In addition, k -means is relatively efficient with time complexity of $O(nmkI)$, where n is the size of a dataset, m is dimensions of a dataset, k is the number of the clusters, and I is the number of iterations. Our proposed adaptive clustering algorithm aims to be more efficient than k -means. The design of the proposed adaptive clustering algorithm is significantly different from the design of k -means. Compared with the traditional k -means clustering, adaptive clustering has the following five novel features, including selective cluster center initialization, flexible number of clusters, adaptive cluster radius, reduced number of rounds to read data, and low-overlap but high-quality cluster formalization. Here the low-overlap means when the clusters are formalized, we try to avoid the cases that a data point has distances to two or more cluster centers less than the radius of these clusters. Having low-overlap feature, the chance of a data point can be represented by two or more cluster centers is lower. This helps reduce the number of final clusters and thus to increase the compression ratio.

The details of the adaptive clustering algorithm is described in Algorithm 1, which consists of three steps: initialization, stabilization, and termination. The first two steps are described in Algorithm 2 and Algorithm 3, while the termination step just contains a condition to decide whether the algorithm should stop. Since the adaptive algorithm is lossy, it will take the maximum information loss as input from the user and treat it as the maximum radius for each cluster. Thus, the information loss will be bounded by the user input to make the retrieved data consistent. Next, we explain each step in the adaptive clustering in detail.

ALGORITHM 1: Adaptive_Clustering()

```

1: Initialization
2: while (!termination) do
3:   Stabilization
4: end while

```

4.2.1 Initialization. The first step in clustering is to determine the number of initial clusters, which is a NP-hard problem (Luxburg 2007), and get a set of initial clusters center. We calculate initial number of clusters, k , based on the feature of the dataset and data-quality requirement, which is specified as a parameter from the application, θ . We define k as $\sqrt{n/(\theta * amplifier)}$, where n is the size of a dataset and *amplifier* is used, because θ is usually a very small number. The above heuristic is defined based on the following observation. A bigger value of θ means that the application has a bigger range to tolerate information loss that usually results in a smaller number of clusters, because θ represents the maximum radius of each cluster. While with more data points, n , there is a potential of getting more number of clusters, k , because more data points may imply a bigger range of data based on our preliminary statistical analysis on several datasets. In this way, we adapt our choice of k based on the application requirement. There are many other heuristics defined in the literature (Jain 2010) to find an optimized value of k . Most proposed heuristics are more complicated than ours and increase the complexity of the algorithm. We prefer to keep the algorithm light and simple. Moreover, we have a different goal in designing the clustering

algorithm compared with other general-purpose k -means algorithms in that the use of clustering in this article is to trade off data quality and energy efficiency. Finally, since k is not fixed value in our design, the initial value of k may not significantly affect the performance of our algorithm. Instead, it can be changed by considering the distance between clusters that affects the quality of clusters, that is, the more separation between clusters, the better clustering quality. In this way, we overcome one drawback of k -means in selecting a proper number of clusters. After initializing the number of clusters, we initialize the clusters. Here, creating clusters is not completely random like that in k -means, but it is done based on the distance from the current cluster to other clusters because we try to make the resulted cluster separated as far as possible. To test whether the new selected cluster is overlapping with other clusters, the *noOverlap* function is used. Here, overlap is defined as the case that a data point has distances to more than two cluster centers less than the radius of these clusters. The *noOverlap* function calculates the distance between the new created cluster and all other existing clusters. Since we limit the maximum radius of each cluster by θ , that distance between any two clusters centers should be at least equal to the summation of their clusters radius, i.e., $\|c_{new} - c_i\| \geq 2\theta \geq r_{new} + r_i$, where c_{new} , r_{new} , c_i , and r_i are the new created cluster center, its radius, any other cluster, and other cluster's radius, respectively. If the calculated distance is more than or equal to 2θ or the sum of the two radius, then the function will return true, which means there is no overlap between the new selected cluster and other clusters. Otherwise, the function will return false. This is to avoid the potential closeness of initial clusters when they are created, which is considered a drawback of k -means clustering, because the more separated the clusters, the better quality of the initial set of clusters. The *noOverlap* function also helps to get better clustering results in the cluster stabilization step.

ALGORITHM 2: Initialization

```

1: procedure CLUSTER_INIT
2:    $k = \sqrt{n}/(\theta * amplifier)$ 
3:   select first cluster randomly;
4:   count=2
5:   while (count  $\leq k$ ) do
6:     newCluster= new random selection from dataset
7:     if (noOverlap (C,newCluster)) then
8:       add newCluster to the list of clusters;
9:       count++
10:    end if
11:  end while
12: end procedure

```

4.2.2 Stabilization. The second step is used to improve the quality of the created clusters, that is, data within the same cluster should have high similarity, while data in different clusters should have very low similarity. Each piece of data will be assigned to one of the initialized clusters or a new cluster based on the following several conditions. First, the cluster that has a shortest Euclidean distance will be picked as a potential cluster to put the data if and only if adding the data to the cluster will not increase the radius of that cluster to be larger than θ and the resulted cluster has no overlap with the other existing clusters. Otherwise, a new cluster will be created containing only these data, and the number of clusters (k) has been changed. The first condition makes sure the resulted clusters are not violating the data-quality requirement, and the second condition guarantees the quality of clusters. This step removes two limitations of k -means clustering algorithm. First, some data that are far from other clusters will not be treated as outliers and filtered. Second,

creating new clusters can reduce the within-cluster sum of square (MacQueen et al. 1967), which helps to improve the clustering result.

After adding the data to the existing or new created cluster, the cluster information will be updated, including the size, the radius, and the center of that cluster. The center of the cluster will be recalculated based on Equation (8),

$$newCenter = \frac{\max_c + \min_c}{2}. \quad (8)$$

If the dataset is a one-dimensional vector, then $\max_c$, $\min_c$, and the cluster center will be represented by a single data value. Otherwise, $\max_c$, $\min_c$, and the cluster center will be represented as an m-tuple, where m is the dimension of the dataset. More specifically, if the dataset is an m-dimension, the max and min of the center will be calculated by Equations (9) and (10), respectively,

$$\max_c = (\max_{t_1}, \max_{t_2}, \dots, \max_{t_m}), \quad (9)$$

$$\min_c = (\min_{t_1}, \min_{t_2}, \dots, \min_{t_m}), \quad (10)$$

where $t_1, t_2, \dots, t_m$ are the data attributes. Instead of updating the cluster center after all data points have been assigned to a cluster as in the k -means clustering algorithm, the cluster center will be updated incrementally when assigning each data point to a cluster. It helps to assign data points to the right nearest cluster, which is calculated from immediate cluster center updating.

4.2.3 Termination. The termination condition defines that a set of high-quality clusters have been found, so it stops stabilization. We define the termination condition as

$$((RMSE_{old} * k_{old}) < (RMSE_{new} * k_{new})) \rightarrow \text{Stop}, \quad (11)$$

where $RMSE_{old}$, k_{old} , $RMSE_{new}$, and k_{new} are RMSE values and the number of clusters before and after this round of stabilization. The termination condition is a heuristic test to conclude whether the current round of stabilization improves the quality of the clusters. We choose the above condition to tradeoff the quality of cluster and compression ratio. We try to balance information loss and compression ratio, because decreasing the value of RMSE means decreasing the information loss, and decreasing the value of k leads to an increase of the compression ratio, which leads to worse data quality.

The details of stabilization step is depicted in Algorithm 3. *inRange* in Algorithm 3 is a function to test whether the current data point is within the radius of cluster.

ALGORITHM 3: Stabilization

```

1: update  $RMSE_{old} = RMSE_{new}$  and  $k_{old} = k_{new}$ 
2: for  $1 \leq i \leq m$  do
3:    $closestCluster \leftarrow \min(\text{distance}(\text{dataset}[i], \text{Centers}))$ 
4:   if ( $\text{inRange}(closestCluster, \text{dataset}[i])$ ) then
5:     if ( $\text{noOverlap}(closestCluster, \text{Centers})$ ) then
6:       put  $closestCluster$  to the list of Clusters;
7:       update by recalculating centre and radius;
8:     end if
9:   else
10:    make  $\text{dataset}[i]$  as center of new cluster;
11:     $k_{new}++$ 
12:    put  $k_{new}$  to the list of Clusters;
13:    calculate newRMSE;
14:   end if
15: end for

```

By using adaptive clustering, the information loss can be bounded by taking the application's data-quality requirement into consideration while the requirement is satisfied in the process of the clustering.

4.3 Cluster Code Generation

The output of the adaptive clustering algorithm will be a list of clusters indicated by the cluster center, the radius, and the number of data points within this cluster. The generated clusters are sorted based on the number of points in the cluster in decreasing order. Then, the Huffman encoding method (Huffman 1952) is applied to generate a code for each cluster. The Huffman encoding scheme takes each symbol and its weight (or frequency of occurrence) and generates proper encodings for each symbol taking into account the weights of each symbol, so higher-weighted symbols have fewer bits in their encoding. In our situation, the symbols are clusters represented by their centers, and the weight of each symbol is the number of data points in its corresponding cluster. Comparing the shortest length of 16 bits to represent an integer, the resulted code can significantly save space in representing the same set of data.

4.4 Encoding with Improvement Approach

With clusters generated using adaptive clustering and code generated using the Huffman encoding method, we are ready to encode the original dataset into a coded set. The data points within the cluster will be represented by the center of the cluster, which is further encoded using the corresponding code for the cluster. Considering data quality, we introduce an improvement approach to further reduce the information loss in this process. Since each data point is represented using the cluster center, the information loss is indicated by the distance between the data point and the cluster center. We try to remember the distance so it can be used later to reduce the information loss when the data are to be recovered at sink. The encoding process is depicted in Algorithm 4.

ALGORITHM 4: Encoding Algorithm

```

1: calculate  $ave = \frac{\sum_i^n dist_i}{n}$ , for  $dist_i > \alpha\theta$ 
2: for each data point  $orig_i$  do
3:   encode it based on its cluster code;
4:   if ( $orig_i > encoded$ ) then
5:     if ( $\|orig_i, (encoded + ave)\| < \|orig_i, encoded\|$ ) then
6:       put "10" at the beginning of encoded;
7:     else
8:       put "0" at the beginning of encoded;
9:     end if
10:  else
11:    if ( $\|orig_i, (encoded - ave)\| < \|orig_i, encoded\|$ ) then
12:      put "11" at the beginning of encoded;
13:    else
14:      put "0" at the beginning of encoded;
15:    end if
16:  end if
17: end for
```

First, we find the value of *ave* that is calculated based on the distances between the original data $orig_i$ and its cluster center. We calculate *ave* as an average distance on each dimension between the cluster center and the set of points, which cause significant information loss because they are located far from their cluster center, that is, $ave = \frac{\sum_i^n dist_i}{n}$, where $dist_i$ is the distance between any data point and its corresponding cluster center, and that distance should be big enough to be

considered. The term “big enough” means the distance is greater than a percentage of θ , that is, $\lambda = \alpha\theta$, where $\alpha \in (0,1)$. For the data points to be included in the calculation of *ave*, its distance, $dist_i$, should be greater than λ .

If the dataset is m -dimensional, which means both *ave* and λ have m attributes, then the comparison is performed on each attribute. With *ave*, we separate the data points within one cluster into two groups. One has data points that have a distance to the center less than *ave*, and the other has data points that have a distance to the center greater than *ave*. For the first group, the max information loss is bounded by *ave* when the data points are represented by the cluster center. There is no need of improvement for this set of points, because the information loss is not significant, and one bit, “0,” will be added at the beginning of code that represents the corresponding encoded data. For the second group data points, if we do not add improvement, then the information loss will be between *ave* and the radius of the cluster. It may be a significant loss, although it still satisfies the application data-quality requirement. We can reduce the information loss by remembering *ave*, that is, we add extra bits, “10” and “11,” to the codes representing data points that are located in the second group. The first bit will be “1” to indicate there is improvement, while the second bit indicates the position of each data point according to the relation to its cluster center. More specifically, for a data point above its cluster center ($datapoint > clustercenter$), the second added bit will be “0”; otherwise, the second bit will be “1.” $datapoint > clustercenter$ is defined as the value of data point at any dimension is larger than the value of the cluster center at the corresponding dimension. For example, for one-dimension data, if a cluster center is c , and a data point is x , then the data point is above its cluster center if $x > c$. For three-dimension data, if the center is $\langle c_1, c_2, c_3 \rangle$ and a data point is $\langle x_1, x_2, x_3 \rangle$, then the data point is above its cluster center only when $x_1 > c_1$, $x_2 > c_2$, and $x_3 > c_3$.

We calculate just one value of *ave* for the whole dataset. Although we can also calculate different *ave* values for different clusters so the retrieved data will be more accurate by using *ave* in decompression, it is necessary to store many *ave* values and send them to sink. This conflicts with our purpose of storing and sending as few bits as possible to save energy. For further improving the quality of the data in terms of accuracy, we can use more bits to remember finer granularity distance to the center, for example, currently we use just one bit to indicate whether the compressed data $> ave$, or $< ave$, while the additional bit can be used to specify finer granularity, such as $1/4$, half, or $3/4$ of the distance to the center. After compression, the value of *ave*, the Huffman encoding tree, and the encoded data are sent to the sink.

4.5 Decompression

After the sink receives the set of encoded data, the decompression will be performed as described in Algorithm 5. By checking the first bit of the received code for a data point, we can know if the data point is located closer to or farther than *ave* to the cluster center. For example, if the first bit of the code is “1,” it is an improved data point, which means that the data point is located farther than *ave* to the cluster center ($|clustercenter - datapoint| > ave$), and thus we need to adjust *ave* to minimize the information loss when the code representing the data point is decompressed. The second bit of the code specifies whether the data point is above the cluster center. The adjustment is performed by subtracting or adding the value of *ave* to the decoded data based on the second bit of the code. If the second bit is “0,” then $newData = decodedData + ave$. Otherwise, $newData = decodedData - ave$.

The rest of the bits represent the cluster code stored in the Huffman encoding tree. Based on all of the above information and the value of cluster center, we are able to retrieve the approximation of the original data point. It is easy to see that this process bounds error because $|\hat{X}_i - X_i| \leq r \leq \theta$.

Table 2. One-Dimensional Dataset Description

	Location	Range (°C)	Samples	Date (mm/dd/yy)	Sampling interval
Set 1	Intel Berkeley Research lab (Bodic et al. 2004)	−20 to +40	25209	02/28/04 to 04/05/04	5 min
Set 2	Paris,France (wea 2015)	+12 to +46	4003	08/16/15 to 09/16/15	10 min
Set 3	Vienna,Austria (wea 2015)	+13 to +42	2854	07/16/15 to 09/16/15	30 min
Set 4	Oklahoma City,OK,USA (Climate Data Online 2010)	−1 to +34	8759	01/01/10 to 12/31/10	60 min
Set 5	New York City,NY,USA (Climate Data Online 2010)	−2 to +29	8759	01/01/10 to 12/31/10	60 min
Set 6	Washington,DC,USA (Climate Data Online 2010)	−3 to +30	8759	01/01/10 to 12/31/10	60 min

Table 3. m-Dimensional Datasets Description

	Location	Samples	Attributes' types	Date (mm/dd/yy)	Sampling interval
Set 7	Oklahoma City,OK,USA (Climate Data Online 2010)	8760×3	Temperature, wind speed, and pressure	01/01/10 to 12/31/10	60 mins
Set 8	Intel Berkeley Research lab (Bodic et al. 2004)	14162×4	Temperature, humidity, light, and voltage	02/28/04 to 04/05/04	31 sec

ALGORITHM 5: Decompression Algorithm

```

1: for i= 1..n do
2:   if (receivedDatai[1] == 1) then
3:     decodedDatai= decode from third bit based on Huffman tree;
4:     if (receivedDatai[2]==1) then
5:       newDatai= decodedDatai- ave
6:     else
7:       newDatai= decodedDatai+ ave
8:     end if
9:   else
10:    newDatai= decode from second bit based on Huffman tree;
11:   end if
12: end for

```

5 PERFORMANCE EVALUATION

In this section, we evaluate the performance of QAAC over several indoor and outdoor real-life datasets. As mentioned previously, our work could be applied for 1 to m-dimensional datasets. We select six one-dimensional datasets as in Table 2 and two m-dimensional datasets as in Table 3. Each dataset is described by six characters, and all datasets have various ranges of readings, sizes,

and intervals. The six datasets in Table 2 contain samples of temperature readings in celsius degrees. Set 1 is collected by the TinyDB in-network query processing systems (Bodic et al. 2004). It contains 25,209 samples that range from -20 to $+40$ and the interval between each two samples is 5 minutes. Set 2 and Set 3 have samples with positive values in deferent ranges and sizes (wea 2015). Set 2 contains 4,003 samples ranging from $+12$ to $+46$ with a 10-minute interval between each two neighboring samples, while Set 3 has 2,854 samples that range from $+13$ to $+42$ with a 30-minute interval between each two neighboring samples. Each of the last three datasets in Table 2 has 8,759 samples with both negative and positive readings, and the interval between each two samples is 60 minutes (Climate Data Online 2010). Set 7 (Climate Data Online 2010) in Table 3 contains 8,760 samples of temperature, wind speed, and pressure data with a sampling rate of every 60 minutes, and Set 8 (Bodic et al. 2004) has 14,162 samples of temperature, humidity, light, and voltage data with a sample rate of once every 31s. These datesets are all real-world datesets containing a set of environmental data with both positive and negative readings at different sampling rates. The diversity in these real-world datesets can test the robustness and reliability of our proposed algorithm.

We compare the performance of QAAC with three robust and popular lossy compression algorithms (Liang and Li 2014; Zordan et al. 2016), including Lightweight temporal compression (LTC) (Schoellhammer et al. 2004), Space Efficient Streaming (SES) (Buragohain et al. 2007), and K run length encoding (K-RLE) (Capo-Chichi et al. 2009), and four lossless compression algorithms, including run length encoding (RLE), Lossless entropy compression (LEC) (Marcelloni and Vecchio 2009), Sensor LZW (S-LZW) (Sadler and Martonosi 2006), and Minimalist, Adaptive and Streaming (MAS) (Assi et al. 2013).

5.1 Performance Metrics

The goal of QAAC is to significantly reduce the amount of transmitted data while maximally preserving the information in the original dataset through an efficient data compression algorithm. Therefore, we will evaluate QAAC in terms of information loss and compression ratio.

RMSE (Guestrin et al. 2004) and MAE (Witten and Frank 2005) have both been used to evaluate the accuracy of the compression algorithms. To evaluate the precision of the compression algorithms, standard deviation has been used. To measure the data reduction performance of compression algorithms, we use compression ratio, which is defined as follows (Campobello et al. 2015):

$$r_c = \left(1 - \frac{\text{\#of bits after compression}}{\text{\#of bits before compression}} \right) * 100\%. \quad (12)$$

When fewer bits are used to represent the original data, it is better for energy conservation. The completeness of data is measured by using entropy based on Equation (4). The lower value of entropy, which means fewer bits used to represent the original data point, is better for energy conservation. To evaluate the consistency of data, we use absolute error based on Equation (6). The lower the absolute error means, the closer the retrieved data are to the original one.

5.2 Experimental Results

5.2.1 Tuning the Parameters in the Proposed Algorithm. One of the parameters is θ that is input by the user to specify the acceptable information loss. Please note that in the evaluation results, the value of θ is shown as none normalized value. When θ is used in the experiments, it is normalized accordingly. The other parameter is ave which is used in the improvement approach. As we mentioned before, adjusting α leads to a proper value of ave which means better data quality. We notice that the best value of α is 0.25 which means 25% of θ . This is because based on our

Table 4. Comparisons of QAAC and Other Lossy Algorithms in Terms of Accuracy on Set 1

θ	LTC			SES			K-RLE			QAAC w/out			QAAC w/improve.		
	Compr.(%)	RMSE	MAE	Compr.(%)	RMSE	MAE	Compr.(%)	RMSE	MAE	Compr.(%)	RMSE	MAE	Compr.(%)	RMSE	MAE
0.1	41	.04	.03	53	.07	.06	21	.04	.03	81	.04	.03	80	.01	.008
0.3	61	0.14	0.11	68	0.2	0.17	53	0.14	0.11	87	0.13	0.11	85	.03	.02
0.5	68	0.25	0.2	72	0.32	0.28	63	0.25	0.2	88	0.2	0.17	87	.06	.04
0.7	71	0.34	0.27	74	0.45	0.39	67	0.34	0.27	90	0.3	0.26	89	.1	.06
1	73	0.49	0.39	76	0.63	0.55	71	0.49	0.39	91	0.41	0.35	90	0.12	.08

Table 5. Comparisons of QAAC and Other Lossy Algorithms in Terms of Completeness on the Six Sets

Sets	Uncompressed	LTC	SES	K-RLE	QAAC
Set 1	9.58	9.49	9.44	9.49	7.39
Set 2	7.37	7.29	7.24	7.29	7.37
Set 3	7.77	7.74	7.71	7.74	7.77
Set 4	9.18	9.18	9.18	9.18	8.67
Set 5	9	9	8.98	9	8.59
Set 6	9.13	9.13	9.12	9.13	8.64

experiment with the selected dataset, the value of RMSE mostly does not exceed $\frac{1}{2}\theta$. We also observe that the best α is $\frac{1}{2}$ RMSE, so $\alpha = \frac{1}{2}$ RMSE $\rightarrow \frac{1}{4}\theta$.

5.2.2 Comparisons with Lossy Compression Algorithms in Terms of Accuracy. Table 4 shows the comparison between QAAC and other lossy compression algorithms over Set 1 in terms of compression ratio and information loss. In Table 4, each lossy algorithm is evaluated by compression ratio %, RMSE, and MAE under the scenario of different values of θ , which indicates the possible maximum information loss that can be tolerated by the application. Since our algorithm is adaptive, different values of θ are used to study the relation between θ and compression ratio. Similarly, other lossy compression algorithms also specify how much the application can tolerate information loss. Here we describe how is used in the compared lossy compression algorithms. In LTC, there is a parameter ϵ that specifies the tolerable margin error. We used as the value of ϵ . In SES, a data stream is converted into a list of segments and is used as the height of each segment, which indicates the error range. Similarly, K defines the maximum information loss of K-RLE; therefore, is used as the value of k . We can see that both versions of QAAC, with and without improvement approach, have much higher compression ratio and lower values of RMSE and MAE compared with other lossy algorithms. For example, the values of RMSE and MAE for QAAC with applying improvement approach are less than $\frac{1}{4}$ of that of all other algorithms for most θ . It indicates that QAAC with approvement approach can really achieve very high accuracy while significantly reducing the amount of data. The basic QAAC algorithm also achieves a much better compression ratio compared with LTC, SES, and K-RLE with better or at least the same RMSEs and MAEs.

5.2.3 Comparisons with Lossy Compression Algorithms in Terms of Completeness. After we showed that our work meets the application requirements with the minimum information loss, which means the result from our work is the most accurate among the results from all other lossy compression algorithm, we now show the comparison between QAAC and other lossy compression algorithms over the six sets in terms of entropy. The results of comparison can be seen in Table 5. In this experiment, we set θ to be 0.1, since the minimum information loss is achieved

when $\theta = 0.1$, as shown in Table 4. We can see that entropy value of QAAC is the same as the original data for Set 2 and Set 3, because in these datasets, QAAC retrieves the original data with zero information loss as we can see in Figures 4(b) and 4(c). QAAC has lower values of entropy compared with other lossy algorithms, which means that it could achieve the maximum compression ratio and retrieve the needed information required by the application at the same time. We also notice that all other lossy compression have almost the same entropy values with uncompressed data, which means they require more bits to represent each data point, so their compression ratio is not that significant.

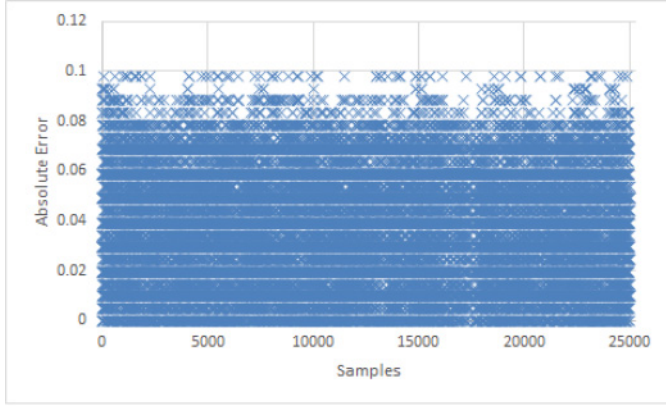
5.2.4 Evaluation in Terms of Consistency. If each decompressed data point satisfies condition 6, then we can say that consistency is maintained. Although consistency can be achieved using QAAC, the degree of consistency can be different. We evaluate QAAC with and without using the improvement approach for the degree of consistency, which is measured using the absolute error and reflects the quality of the received data. Figures 2, 3, 4, and 5 show the result of absolute error over six sets from our work without and with using the improvement approach, respectively. The y -axis of each subfigure in those figures represents the value of absolute error between each data point results from our work and that from the original dataset, while the x -axis represents the number of samples of that dataset. In this evaluation, we set $\theta = 0.1$, which means the maximum absolute error between any two points should not exceed 0.1 to say the result is consistent based on condition 6. When we apply the improvement approach in our work, we set the value of $\alpha = 0.25$, and $\lambda = \alpha \times \theta \rightarrow 0.25 \times 0.1$, so $\lambda = 0.025$.

The degree of consistency depends on the dataset itself, for example, when using our work without improvement on Set 1, as we can see in Figure 2(a), the max absolute error is 0.098, while the max absolute error is 0.05 when we apply our work without improvement on Set 3, as is shown in Figure 2(c).

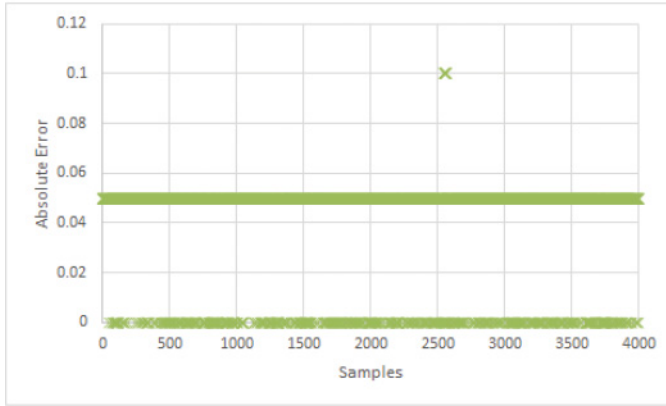
Comparing our work with and without using the improvement approach, we indicate that the absolute error of our work when applying the improvement approach is much less than that from our work without applying that approach, as we can see in Figures 4 and 5. For example, the absolute error of our work when applying the improvement approach is less than half of that from our work without applying that approach on Set 1 and Set 2 as we can see in Figures 4(a) and 4(b). The best value we could achieve of the absolute error is 0, which means that we could retrieve that exact value of original data that when applying our work with using the improvement approach on Set 3, as is shown in Figure 4(c). Based on that, we can conclude that both of our work with and without improvement are consistent, but with improvement is much more consistent than without that approach. The absolute errors look close for sets 2 to 6, but it may be because there are so many samples, so the details of the difference cannot be visualized. It can also be because our algorithm find a set of good cluster centers to represent many data points.

5.2.5 Comparison with Lossy Compression Algorithms over One-Dimensional Datasets in Terms of Compression Ratio and Execution Time. Figures 6 and 7 show the comparison between QAAC and three lossy compression algorithms for all si datasets in terms of compression ratio and execution time. The x -axis of each subfigure specifies values of the acceptable information loss, θ , and the y -axis shows compression ratio.

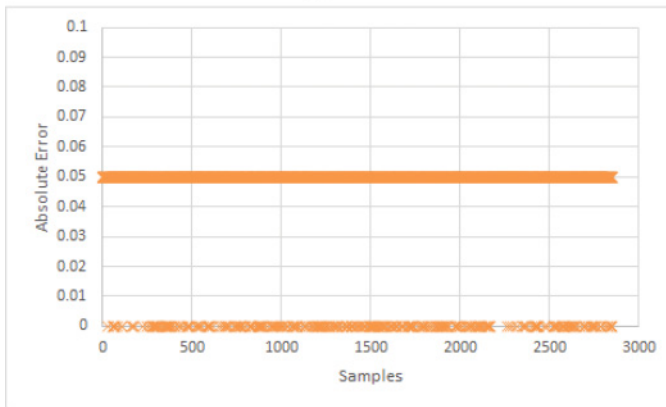
From the figures, we observe that the data compression ratio increases with the increasing of θ . For all datasets, QAAC has the maximum data compression ratio. It over-performs other lossy compression algorithms especially when θ is small, that is, QAAC can achieve both goals of high data compression ratio and very small information loss better than other lossy compression algorithms. For different datasets, the compression ratio is higher for datasets Set 1 and Set 2 for most algorithms, and it is lower for other datasets, Set 3, Set 4, and Set 6, but QAAC achieves a fairly



(a) Set 1

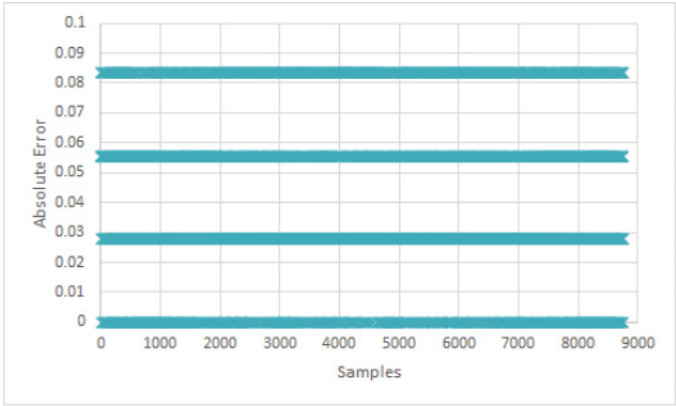


(b) Set 2

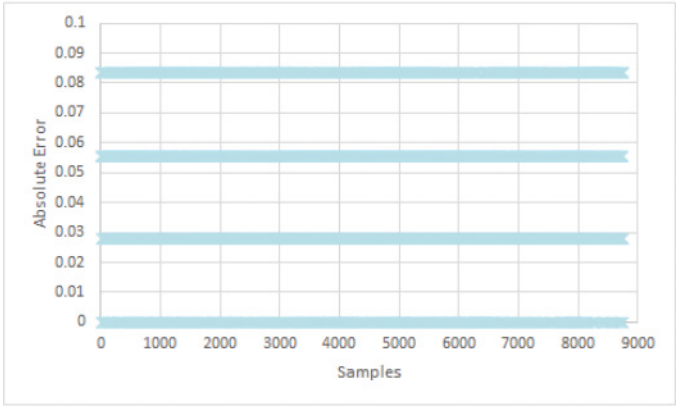


(c) Set 3

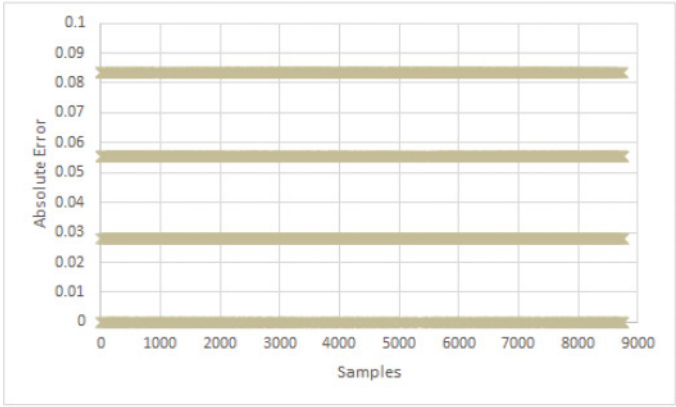
Fig. 2. Absolute Error between each Data Point retrieved by QAAC without using Improvement Approach and the Original Data from Sets 1–3.



(a) Set 4

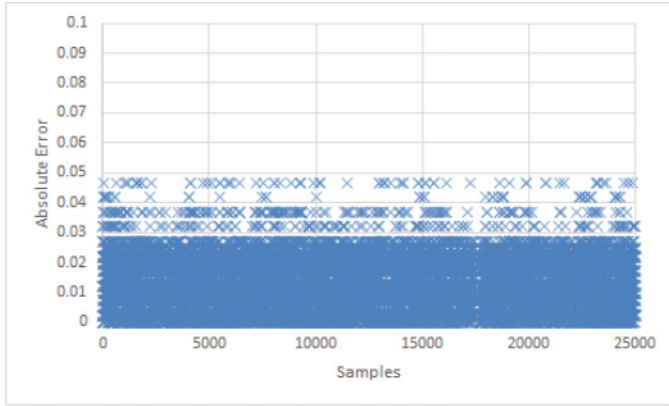


(b) Set 5

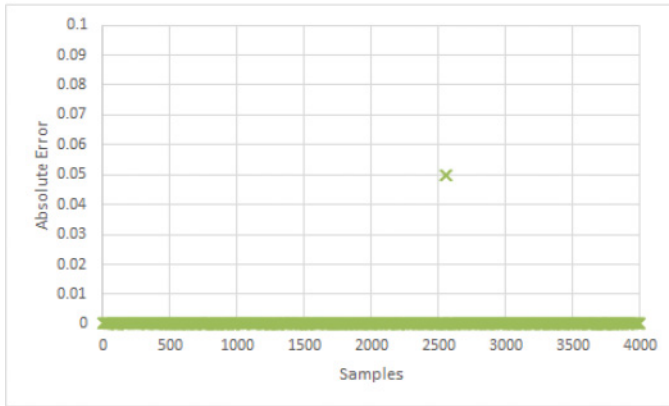


(c) Set 6

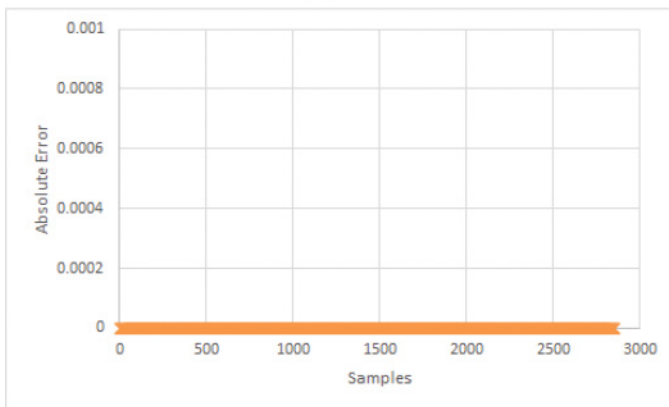
Fig. 3. Absolute Error between each Data Point retrieved by QAAC without using Improvement Approach and the Original Data from Sets 4–6.



(a) Set 1

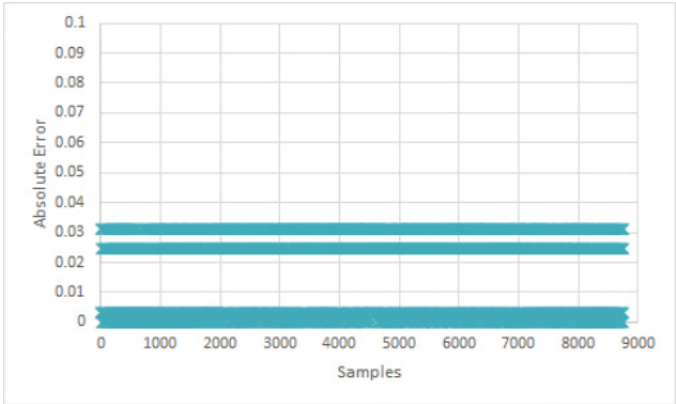


(b) Set 2

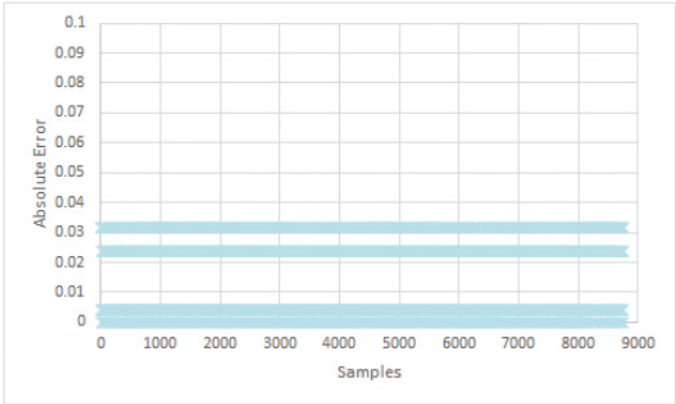


(c) Set 3

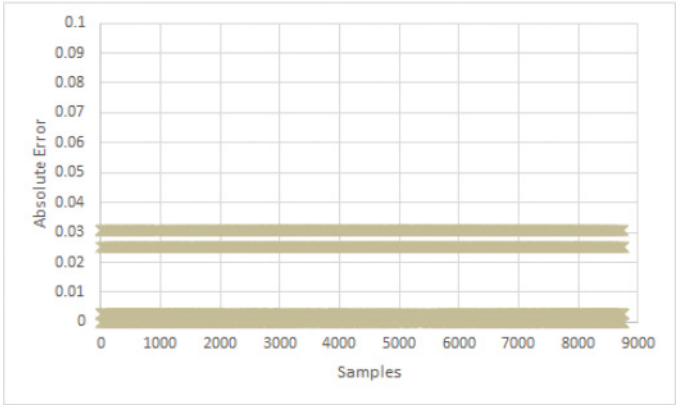
Fig. 4. Absolute Error between each Data Point retrieved by QAAC with using Improvement Approach and the Original Data from Sets 1–3.



(a) Set 4

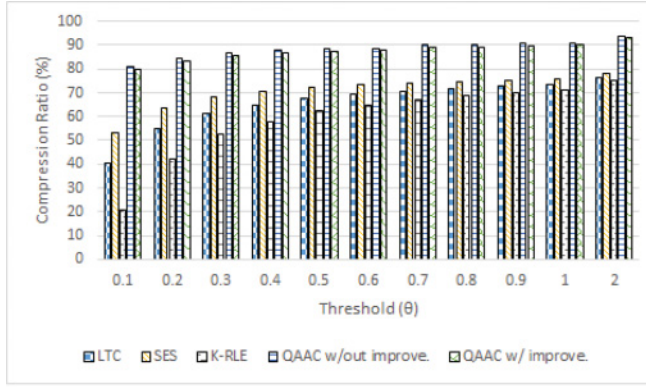


(b) Set 5

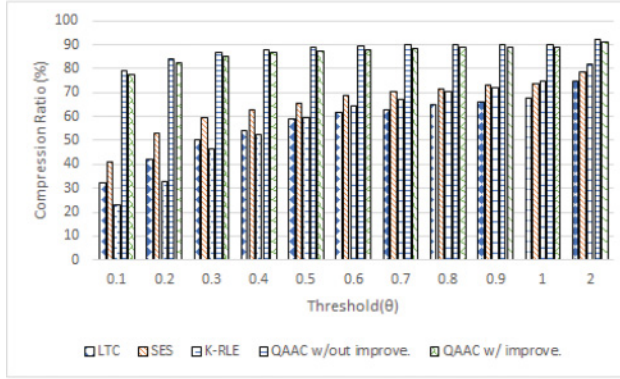


(c) Set 6

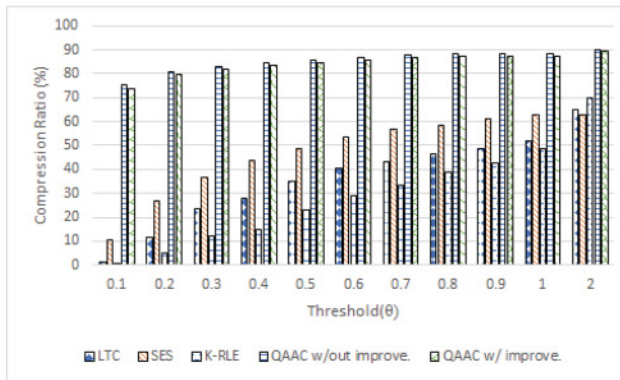
Fig. 5. Absolute Error between each Data Point retrieved by QAAC with using Improvement Approach and the Original Data from Sets 4–6.



(a) Set 1

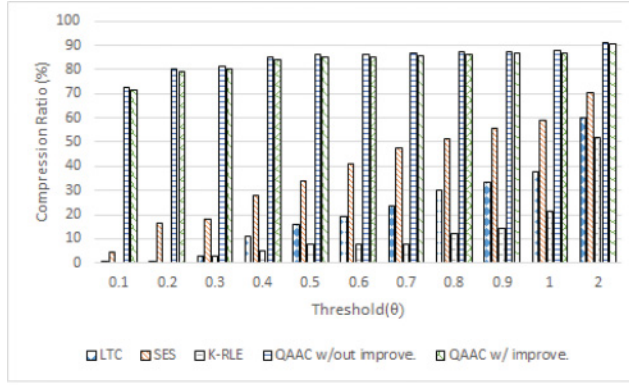


(b) Set 2

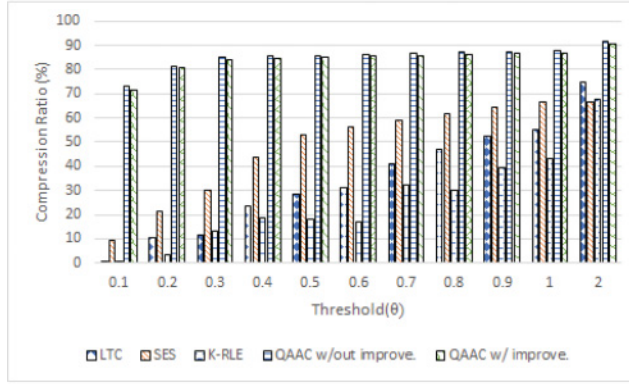


(c) Set 3

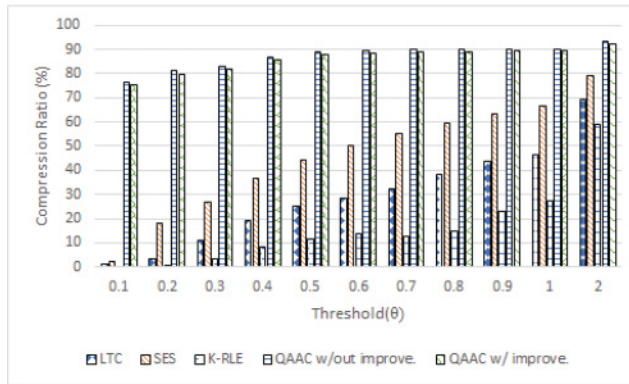
Fig. 6. QAAC vs. Lossy Compression Algorithms in terms of compression ratio for different (θ) (Sets 1–3).



(a) Set 4



(b) Set 5



(c) Set 6

Fig. 7. QAAC vs. Lossy Compression Algorithms in terms of compression ratio for different (θ) (Sets 4–6).

Table 6. Entropy Values and Max Lossless Compression Ratio Can Be Achieved from the Six Sets

Sets	Entropy (bit)	Max Compression (%)
Set 1	9.58	70%
Set 2	7.37	77%
Set 3	7.77	76%
Set 4	9.18	71%
Set 5	9	72%
Set 6	9.13	71%

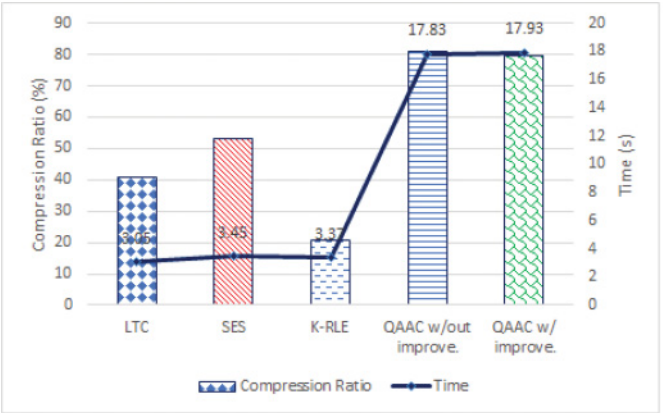
high compression ratio with the lowest of above 70% in all datasets, while K-RLE only achieves 1% compression ratio for Sets 3 to 6. This is because increasing the sampling interval from 5 or 10 minutes to 30 or 60 minutes in these datasets significantly decreases the correlation among the sensor readings; it hurts the performance of compression algorithms that achieve data compression based on the correlation. The above observation indicates that QAAC has much better and stable performance compared with other lossy compression algorithms. Among LTC, SES, and K-RLE, K-RLE has a relatively worse compression ratio.

A precision degree of the data has been calculated based on standard deviation. We notice that all the algorithms have almost the same value of standard deviation of the original data. Even if there is a difference between their standard deviation values, that difference is negligible.

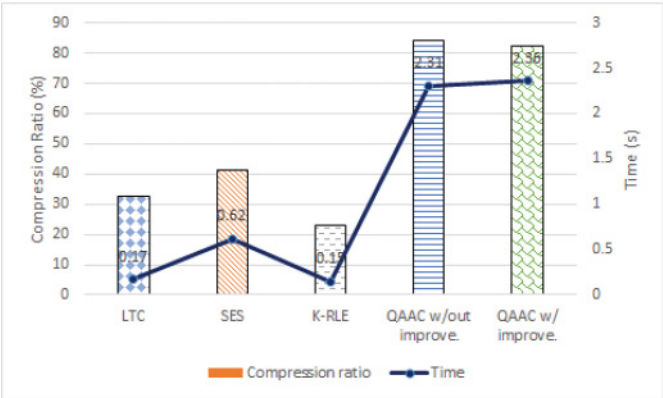
Figures 8 and 9 show comparison of QAAC with other lossy compression algorithms in terms of compression ratio and execution time. The bars in each figure indicate compression ratio, while the line indicates execution time. The experiment is done by setting θ at 0.1. From the figures, we observe that the execution time depends on the size of the datasets. It increases when the dataset size increases. As shown in Figures 8 and 9, the compression ratio is positively correlated with execution time; the higher the compression ratio, the higher the execution time, because it requires more processing to complete compression. As mentioned previously, the most important concern about WSNs is energy, and energy consumption is mainly in data transmission, that is, the energy consumption by sending one bit is equal to the execution of thousands lines of code (Razzaque et al. 2013). Although QAAC has slightly higher time complexity than the others, when we spend more time (more energy) in local processing, we can save a lot energy by transmitting a smaller amount of data.

5.2.6 Performance Comparison with Lossless Compression Algorithms over One-Dimensional Datasets. Figures 10 and 11 show the comparison between QAAC and four lossless compression algorithms for all six datasets in Table 2 in terms of compression ratio and execution time. The bars in each figure indicate compression ratio, while the line indicates execution time. From the figures, we observe a similar result like what we have seen in Figures 6 and 7. It indicates that QAAC has much better and stable performance compared with other lossless compression algorithms. For example, in Set 1, the compression ratio of QAAC is 54, 6, 5, and 2 times the compression ratio of RLE, MAS, S-LZW, and LEC, respectively. Among RLE, LEC, S-LZW, and MAS, RLE has a relatively lower compression ratio. The figures show that to compress the same dataset, QAAC takes more time than LEC, RLE, and MAS, but it takes less time than S-LZW.

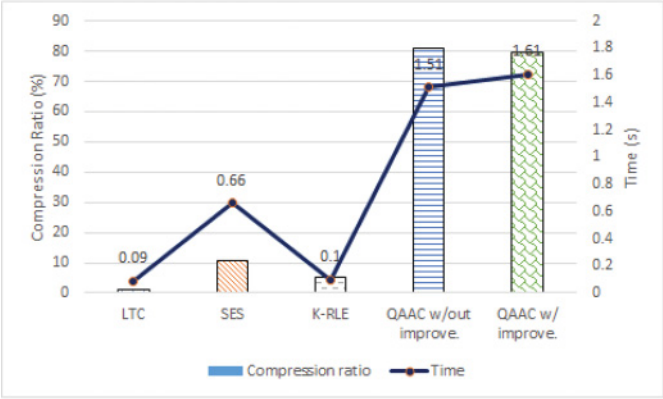
As we have mentioned before, entropy theory can be used to calculate the maximum lossless compression ratio that can be achieved. Based on Equation (5), we calculate the entropy value and maximum compression ratio for each of the six sets. The results are displayed in Table 6.



(a) Set 1

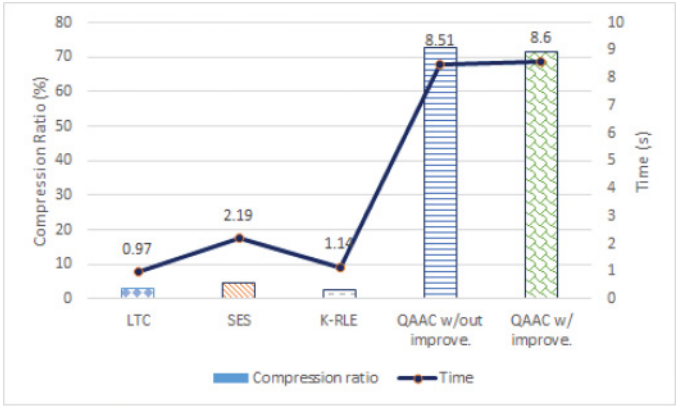


(b) Set 2

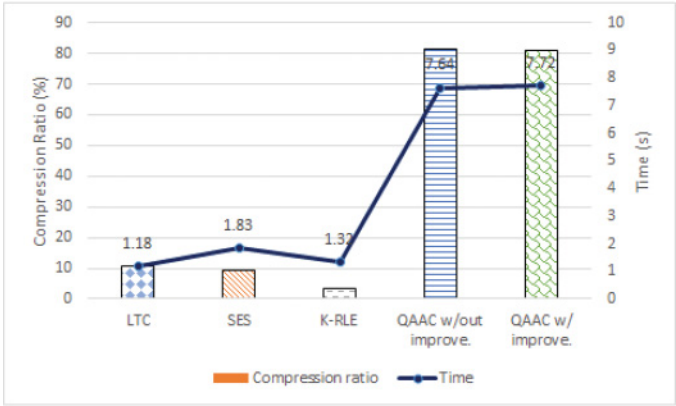


(c) Set 3

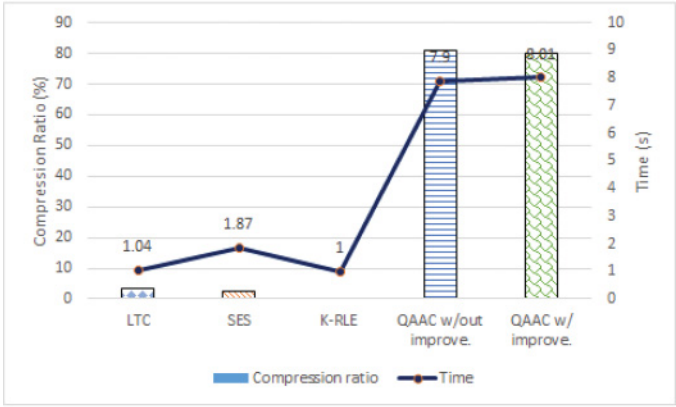
Fig. 8. QAAC vs. Lossy Compression Algorithms in terms of Compression Ratio and Execution Time (Sets 1–3).



(a) Set 4



(b) Set 5



(c) Set 6

Fig. 9. QAAC vs. Lossy Compression Algorithms in terms of Compression Ratio and Execution Time (Sets 4–6).

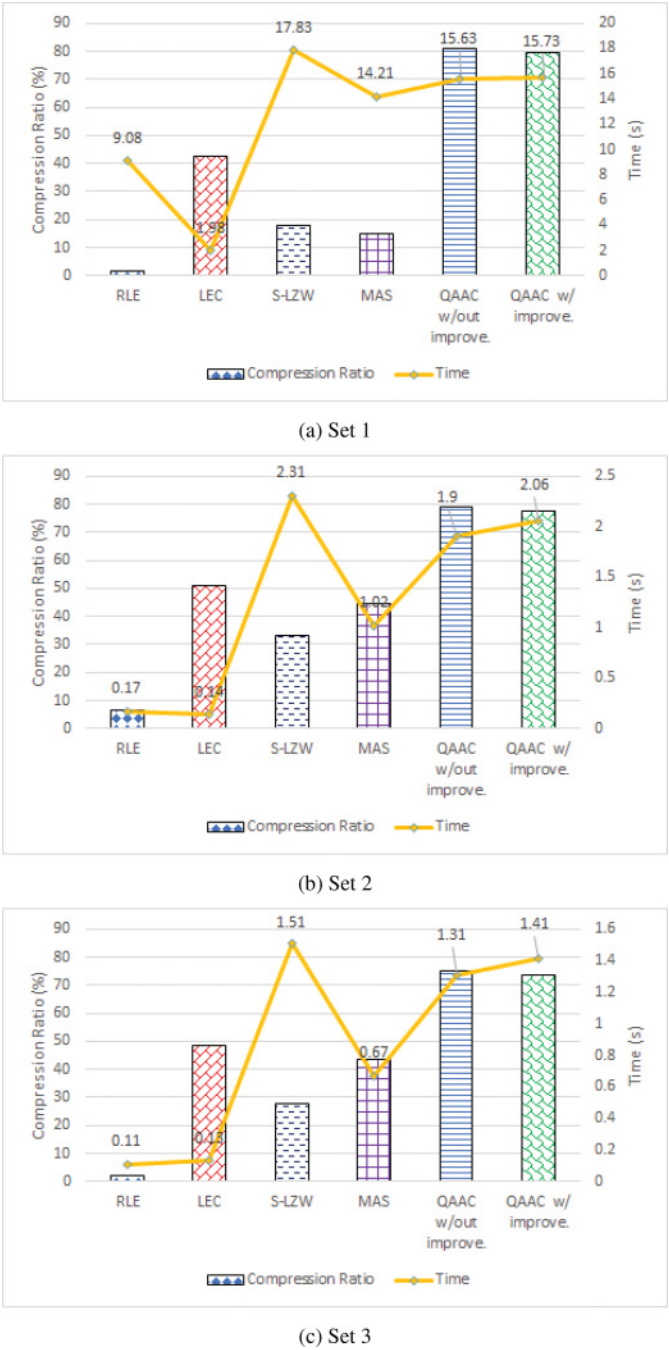
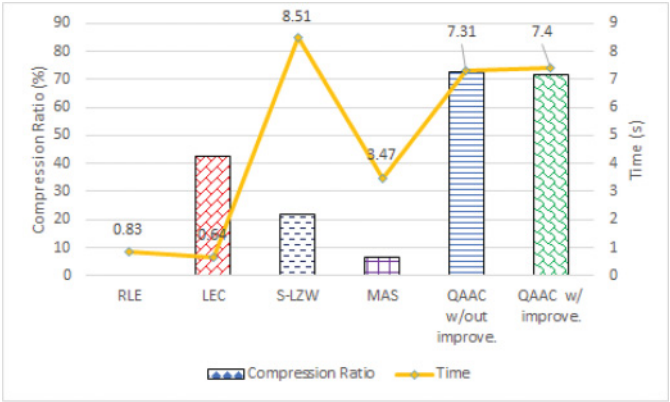
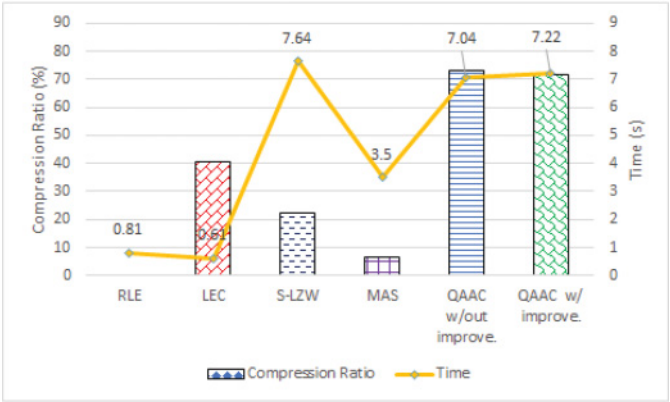


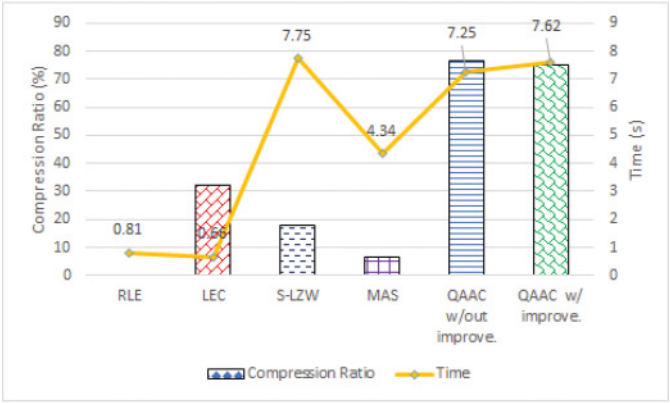
Fig. 10. QAAC vs. Lossless Compression Algorithms in terms of Compression Ratio and Execution Time (Sets 1–3).



(a) Set 4

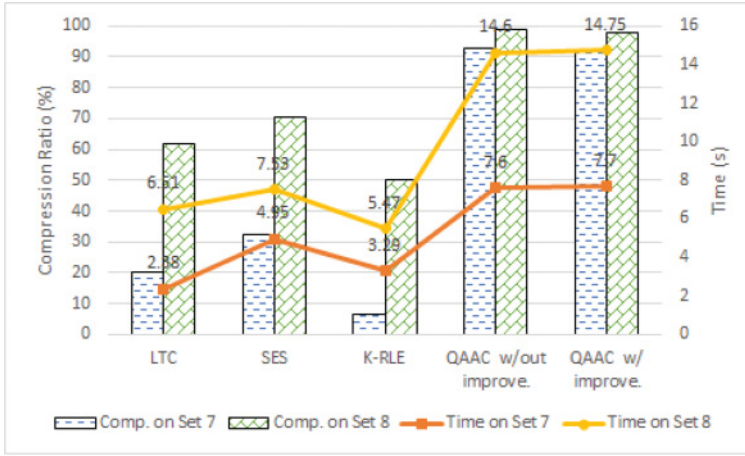


(b) Set 5

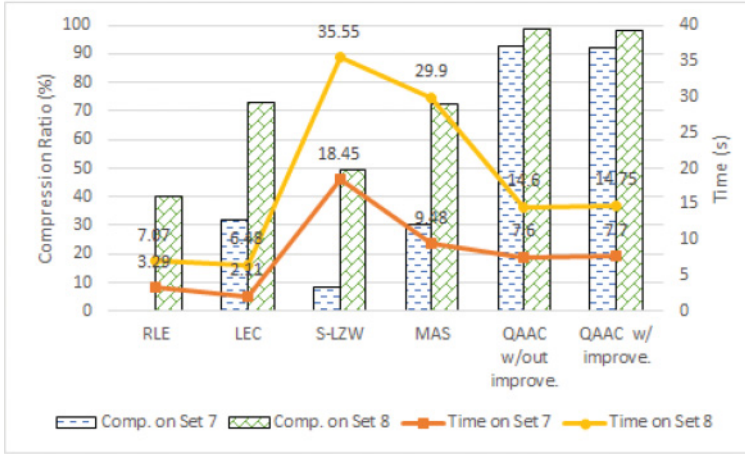


(c) Set 6

Fig. 11. QAAC vs. Lossless Compression Algorithms in terms of Compression Ratio and Execution Time (Sets 4–6).



(a) Lossy algorithms



(b) Lossless algorithms

Fig. 12. QAAC vs. Lossy or Lossless Algorithms in terms of Compression Ratio and Execution Time on Sets 7 and 8.

Comparing the maximum compression ratios from Table 6 and the compression ratios that are achieved by lossless compression algorithms, as shown in Figures 10 and 11, all the lossless compression algorithms used in these experiments could not achieve more than 66% of the possible maximum compression ratio. We can see that LEC has the max compression ratio among RLE, LEC, S-LZW, and MAS, and its compression ratio, for example, in Set 1 is 42%, while the maximum compression ratio is 70%. On the other hand, our work, since it allows for little information loss with $\theta = 0.1$, achieves 80% on Set 1.

5.2.7 Performance Comparison with Lossy and Lossless Compression Algorithms over m-Dimensional Datasets. Figure 12 shows the performance of QAAC compared with other lossy or lossless compression algorithms on Set 7 and Set 8. The bars in the figure indicate compression ratio while the lines in the figure depict execution time.

Table 7. Entropy Values and Max Lossless
Compression Ratio Can Be Achieved
from Set 7 and Set 8

Sets	Entropy (bit)	Max Compression (%)
Set 7	5.55	83 %
Set 8	4.05	87 %

The performance comparison between QAAC and other algorithms exhibits a similar result as they are applied in one-dimensional data. All other algorithms have a better performance when applied in Set 8 compared with applied in Set 7, because the interval of two sensor readings is bigger in Set 7, but QAAC exhibits a stable performance in both datasets. Compared with lossless compression algorithms, QAAC still takes more time to compress the same dataset than RLE and LEC, but the difference is much smaller than that in compression of one-dimensional datasets. In addition, QAAC takes less time than MAS when applied in m-dimensional datasets, although it takes more time when applied in one-dimensional datasets. A similar pattern has been seen for lossy compression algorithms, as shown in Figure 12(a). This shows that QAAC is suitable to be applied in m-dimensional dataset compression, while others are not.

Based on Equation (5), we calculate the entropy value and maximum compression ratio for Set 7 and Set 8 in Table 7. Comparing the maximum compression ratios from Table 7 and the compression ratios that are achieved by lossless compression algorithms, as shown in Figure 12(b), we can see that LEC has the max compression ratio among RLE, LEC, S-LZW, and MAS, and its compression ratios on Set 7 and Set 8 are 32% and 73%, while the possible maximum compression ratios that could be achieved are 83% and 87%. On the other hand, our work, since it allows for little information loss with $\theta = 0.1$, achieves 92% and 98% on Set 7 and Set 8, respectively.

6 RELATED WORK

Many data compression techniques (Aboelela 2014; Yacoab et al. 2010) have been proposed to save energy by reducing the amount of transmitted data. Srisooksai et al. (2012) classify data compression mechanisms into two broad classes, distributed and local, for dense and sparse networks. The distributed data compression approach exploits the high spatial correlation in data from sensors in dense networks (Incebacak et al. 2015). Example compression algorithms designed based on the above principle include distributed source coding, distributed transform coding, distributed source modeling, and compressed sensing. We consider local data compression techniques that usually exploit temporal correlation of the data and do not depend on the specific WSN topologies. Local compression techniques have two classes, lossy and lossless. Lossless compression algorithms based on dictionary require extensive use of RAM, so they are not suitable to be applied directly in WSN due to the limited RAM, or they have to be modified to cater WSN applications. Deepu, Heng, and Liang propose a hybrid data compression scheme in Deepu et al. (2017). In their work, the data are compressed using both lossy and lossless techniques. Like ours, the proposed work also supports adaptive data rate selection and aims to save data transmission power.

S-LZW (Sadler and Martonosi 2006) is a modified version of the popular LZW (Welch 1984) data compression algorithm that is a dictionary-based algorithm. S-LZW uses a 512-entry dictionary to solve the dictionary size problem. When that dictionary gets full, it could be fixed and used as-is or could be reset. S-LZW has to balance four major inter-related parameters, the block size, the dictionary size, the strategy to follow when the dictionary is full, and the mini-cache size. The values of these four parameters deeply influence compression performance, so they have to be fine tuned before deploying S-LZW in sensor nodes (Marcelloni and Vecchio 2009). In Trimming the

tree (Reinhardt et al. 2010), sensor measurements are coded using adaptive Huffman encoding, but to save memory the number of symbols present in the Huffman tree is limited to the measurements that happen most frequently. LEC (Marcelloni and Vecchio 2009) is another dictionary-based algorithm whose size depends on the precision of the analog-to-digital converter, which is used in the compression process. Basically, for each new sensor measurement x_i , the residue $r_i = x_i - x_{i-1}$ is calculated and encoded by a binary sequence based on the dictionary. The algorithm is static. Hence, the algorithm cannot adapt to changes in the source data statistics. The Adaptive Linear Filtering Compression algorithm (Kiely et al. 2010) uses an adaptive linear filter to predict the future M samples of the dataset, and the prediction errors are compressed using an entropy encoder. To work with sensor nodes with limited computational capability, the method employs a quantization mechanism. MAS compression algorithm (Assi et al. 2013) is a lossless compression algorithm for floating-point data with a single-precision. This algorithm requires the numbers to satisfy two conditions to be compressed. First, if a floating-point number exceeds seven significant digits it would be truncated. Second, the floating-point number when put in scientific notation must have a power of 32 or less. The first step in the compression process is to convert each number into scientific notation. Then each number will be encoded based on the proposed format: $nnn|eeee|ns|es| \dots$ where the first 3 bits (nnn) define the number of digits, the next 5 bits ($eeee$) specify the exponent, ns is the sign of significance, es is the sign of exponent, and the rest 4 to 24 bits represent the integer form of the whole number without point. Based on this encoding technique, a minimum number of 14 bits are used to encode each floating point number.

There are many lossy compression algorithms in the literature, too. The LTC algorithm (Schoellhammer et al. 2004) is designed to find a piecewise linear representation for time series in sensor data. They assume environmental data as temporally continuous and approximately linear, so they use linear approximation to compress the data. The compression is done by replacing each line segment using the first point of the line segment. Another example is SES (Buragohain et al. 2007) that converts a data stream into a list of segments where each segment has a height of 2θ , where θ is the user input to be considered as maximum error. Each segment is represented by three parameters: t_s , t_e , and v_0 , where t_s and t_e are timestamps of first and last data points in the segment, and v_0 is the minimum data point in the segment. An incoming data point is greedily packed into the latest segment if not violating the definition of segment; otherwise, a new segment is created. A lossy compression algorithm is also proposed in Wu et al. (2016). The authors use data prediction, compression, and recovery to reduce the communication cost while guaranteeing the accuracy of the collected data. Data prediction is achieved by implementing the Least Mean Square dual prediction algorithm and by adding an optimal step of minimizing the mean-square derivation, where the cluster heads can obtain a good approximation of the real data from the sensor nodes. There are also a large class of lossy data compression techniques that transform the raw data into other domains. Examples include discrete Fourier transforms, fast Fourier transforms (Zordan et al. 2012), and discrete cosine transforms (Quer et al. 2009). However, such algorithms have poor performance when used to compress spatial data or noisy data (Alsheikh et al. 2014).

One more example of lossy compression is K-RLE (Capo-Chichi et al. 2009), which is a variation of the RLE method. K-RLE approximates a sequence of n data points with values in the range of $[d - K, d + K]$ as the pair (n, d) , where K defines the maximum information loss of the method and d is the value representing all data points in the range. If $K = 0$, then K-RLE is the same as RLE. Above lossy algorithms perform poorly if the sensor readings fluctuate frequently, even when the fluctuations follow some fixed patterns.

7 CONCLUSION AND FUTURE WORK

This article defines quality-aware data compression problem and proposes QAAC, which considers application's data quality and bounds information loss by using adaptive clustering and novel coding algorithm. The performance evaluation shows that QAAC over performs the existing lossy and lossless algorithms in terms of compression ratio. It is also more efficient than most lossless compression algorithms and achieves better accuracy than all lossy ones.

We plan to extend this work in two ways. First, we will implement and test the proposed algorithm in real sensor systems to validate the performance of the algorithm. Second, we will keep improving the performance of the algorithm and convert it to an online version, which builds clusters and encodes each incoming datum at the time the data are received.

REFERENCES

- E. Aboelela. 2014. LiftingWiSe: A lifting-based efficient data processing technique in wireless sensor networks. *Sensors* 14, 8 (2014), 14567–14585.
- I. Akyildiz, W. Su, Y. Sankarasubramaniam, and E. Cayirci. 2002. Wireless sensor networks: A survey. *Computer Networks* 38, 4 (2002).
- Jamal N. Al-Karaki and Ahmed E. Kamal. 2004. Routing techniques in wireless sensor networks: A survey. *IEEE Wireless Commun.* 11, 6 (2004), 6–28.
- M. Abu Alsheikh and others. 2014. Efficient data compression with error bound guarantee in wireless sensor networks. In *Proceedings of the 17th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM'14)*.
- M. El Assi, A. Ghaddar, S. Tawbi, and G. Fadi. 2013. Resource-efficient floating-point data compression using MAS in WSN. *Int. J. Ad Hoc, Sens. Ubiqu. Comput.* 4, 5 (2013).
- K. Balakrishnan and N. Toubia. 2007. Relationship between entropy and test data compression. *IEEE Trans. Comput.-Aid. Des. Integr. Circ. Syst.* 26, 2 (2007), 386–395.
- R. Blake and P. Mangiameli. 2011. The effects and interactions of data quality and problem complexity on classification. *J. Data Inf. Quality* 2, 2 (2011).
- MPRMote Processor Radio Board and MIBMote Interface. 2003. Programming Board Users Manual. (2003).
- Peter Bodic, Wei Hong, Carlos Guestrin, Sam Madden, Mark Paskin, and Romain Thibaux. 2004. Intel Lab Data. Retrieved from <http://db.csail.mit.edu/labdata/labdata.html/>.
- C. Buragohain, N. Shrivastava, and S. Suri. 2007. Space efficient streaming algorithms for the maximum error histogram. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE'07)*.
- G. Campobello, O. Giordano, A. Segreto, and S. Serrano. 2015. Comparison of local lossless compression algorithms for wireless sensor networks. *J. Netw. Comput. Appl.* 47 (2015).
- E. Capo-Chichi, H. Guyennet, and J. Friedt. 2009. K-RLE: A new data compression algorithm for wireless sensor network. In *Proceedings of the International Conference on Sensor Technologies and Applications*.
- C. Cappiello and others. 2009. Quality-and energy-aware data compression by aggregation in wsn data streams. In *Proceedings of the IEEE International Conference on Pervasive Computing and Communications (PerCom'09)*.
- Q. Chi and others. 2014. A reconfigurable smart sensor interface for industrial WSN in IoT environment. *IEEE Trans. Industr. Inform.* 10, 2 (2014).
- International Electrotechnical Commission and others. 2007. *International Vocabulary of Metrology-Basic and General Concepts and Associated Terms: (VIM)*. ISO Copyright Office.
- Chacko J. Deepu, Chun-Huat Heng, and Yong Lian. 2017. A hybrid data compression scheme for power reduction in wireless sensors for IoT. *IEEE Trans. Biomed. Circ. Syst.* 11, 2 (2017), 245–254.
- E. Fasolo, M. Rossi, J. Widmer, and M. Zorzi. 2007. In-network aggregation techniques for wireless sensor networks: A survey. *IEEE Wireless Commun.* 14, 2 (2007).
- Climate Data Online. 2010. Retrieved from <http://www.ncdc.noaa.gov/cdo-web/>.
- C. Guestrin and others. 2004. Distributed regression: An efficient framework for modeling sensor network data. In *Proceedings of the ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN'04)*.
- Benjamin T. Hazen, Christopher A. Boone, Jeremy D. Ezell, and L. Allison Jones-Farmer. 2014. Data quality for data science, predictive analytics, and big data in supply chain management: An introduction to the problem and suggestions for research and applications. *International Journal of Production Economics* 154 (2014), 72–80.
- David A. Huffman. 1952. A method for the construction of minimum-redundancy codes. In *Proceedings of the IRE* 40, 9 (1952), 1098–1101.

- D. Incebacak, R. Zilan, B. Tavli, J. Barcelo-Ordinas, and J. Garcia-Vidal. 2015. Optimal data compression for lifetime maximization in wireless sensor networks operating in stealth mode. *Ad Hoc Netw.* 24 (2015).
- A. Jain. 2010. Data clustering: 50 years beyond K-means. *Pattern Recogn. Lett.* 31, 8 (2010), 651–666.
- S. Jerez and others. 2013. A multi-physics ensemble of present-day climate regional simulations over the iberian peninsula. *Clim. Dynam.* 40, 11–12 (2013).
- A. Kiely and others. 2010. Adaptive linear filtering compression on realtime sensor networks. *Comput. J.* (2010).
- J. Gana Kolo, S. Anandan Shanmugam, David Wee Gin Lim, Li-Minn Ang, and Kah Phooi Seng. An adaptive lossless data compression scheme for wireless sensor networks. *Journal of Sensors.* (2012), 1–20.
- T. Kwon and J. Cioffi. 2013. Random deployment of data collectors for serving randomly-located sensors. *IEEE Trans. Wireless Commun.* 12, 6 (2013).
- W. Lang and J. Wilkerson. 2008. Accuracy vs. validity, consistency vs. reliability, and fairness vs. absence of bias: A call for quality. (unpublished)
- Yao Liang and Yimei Li. 2014. An efficient and robust data compression algorithm in wireless sensor networks. *IEEE Commun. Lett.* 18, 3 (2014), 439–442.
- Jie Lin, Wei Yu, Nan Zhang, Xinyu Yang, Hanlin Zhang, and Wei Zhao. 2017. A survey on internet of things: Architecture, enabling technologies, security and privacy, and applications. *IEEE IoT J.* pp, 99 (2017), 1–1.
- U. Von Luxburg. 2007. A tutorial on spectral clustering. *Stat. Comput.* 17, 4 (2007).
- J. MacQueen and others. 1967. Some methods for classification and analysis of multivariate observations. In *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability*, Vol. 1.
- Claudio S. Malavenda, Francesco Menichelli, and Mauro Olivieri. 2016. Narrowband delay tolerant protocols for WSN applications: Characterization and selection guide. In *Proceedings of the International Conference on Applications in Electronics Pervading Industry, Environment and Society*. Springer, 109–121.
- F. Marcelloni and M. Vecchio. 2009. An efficient lossless compression algorithm for tiny nodes of monitoring wireless sensor networks. *The Computer Journal* 52, 8 (2009), 969–987.
- G. Quer and others. 2009. On the interplay between routing and signal representation for compressive sensing in wireless sensor networks. In *Proceedings of the Information Theory and Applications Workshop*.
- R. Rajagopalan and P. Varshney. 2006. Data aggregation techniques in sensor networks: A survey. (2006).
- M. Razzaque, C. Bleakley, and S. Dobson. 2013. Compression in wireless sensor networks: A survey and comparative evaluation. *ACM Trans. Sens. Netw.* 10, 1 (2013).
- A. Reinhardt and others. 2010. Trimming the tree: Tailoring adaptive huffman coding to wireless sensor networks. In *Wireless Sensor Networks*.
- C. Sadler and M. Martonosi. 2006. Data compression algorithms for energy-constrained devices in delay tolerant networks. In *Proceedings of the ACM Conference on Embedded Networked Sensor Systems (Sensys'06)*.
- N. Savage and others. 2013. Air quality modelling using the met office unified model (AQUUM OS24-26): Model description and initial evaluation. *Geosci. Model Dev.* 6, 2 (2013).
- K. Sayood. 2012. *Introduction to Data Compression*. Newnes.
- Tom Schoellhammer, Ben Greenstein, Eric Osterweil, Michael Wimbrow, and Deborah Estrin. 2004. Lightweight temporal compression of microclimate datasets. *Center for Embedded Network Sensing* (2004).
- K. Sha and W. Shi. 2008. Consistency-driven data quality management of networked sensor systems. *J. Parallel Distrib. Comput.* 68, 9 (2008).
- K. Sha and S. Zeadally. 2015. Data quality challenges in cyber-physical systems. *J. Data Inf. Qual.* 6, 2 (2015).
- C. Shannon. 2001. A mathematical theory of communication. *ACM SIGMOBILE Mobile Comput. Commun. Rev.* 5, 1 (2001), 3–55.
- V. Shnayder and others. 2004. Simulating the power consumption of large-scale sensor network applications. In *International Conference on Embedded Networked Sensor Systems*.
- T. Srisooksai, K. Keamarungsi, P. Lamsrichan, and K. Araki. 2012. Practical data compression in wireless sensor networks: A survey. *J. Netw. Comput. Appl.* 35, 1 (2012).
- N. Visalakshi and K. Thangavel. 2009. Impact of normalization in distributed k-means clustering. *Int. J. Soft Comput.* 4, 4 (2009).
- C. Wang, J. Shih, B. Pan, and T. Wu. 2014. A network lifetime enhancement method for sink relocation and its analysis in wireless sensor networks. *IEEE Sens. J.* 14, 6 (2014).
- R. Wang, V. Storey, and C. Firth. 1995. A framework for analysis of data quality research. *IEEE Trans. Knowl. Data Eng.* 7, 4 (1995).
- Wea 2015. Weather Underground. Retrieved from <http://www.wunderground.com>.
- T. Welch. 1984. A technique for high-performance data compression. *Computer* 6, 17 (1984).
- Jesse Ray Whitehead. 2016. Cluster-based trust proliferation and energy efficient data collection in unattended wireless sensor networks with mobile sinks. Masters Theses (2016).

- I. Witten and E. Frank. 2005. *Data Mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann.
- Mou Wu, Liansheng Tan, and Naixue Xiong. 2016. Data prediction, compression, and recovery in clustered wireless sensor networks for environmental monitoring applications. *Inf. Sci.* 329 (2016), 800–818.
- M. Yacoab, V. Sundaram, and A. Thajudeen. 2010. A cost effective compressive data aggregation technique for wireless sensor networks. *International Journal of Ad hoc, Sensor & Ubiquitous Computing (IJASUC)* 1 (2010), 116–130.
- Wei Yu, Thang Nam Le, Dong Xuan, and Wei Zhao. 2004. Query aggregation for providing efficient data services in sensor networks. In *Proceedings of the IEEE International Conference on Mobile and Ad-hoc Sensor Systems (MASS'04)*.
- Difan Zhang, Linqiang Ge, Wei Yu, Rommie Hardy, Robert J. Reschly, and Hanlin Zhang. 2013. On effective data aggregation techniques in hostbased intrusion detection in MANET. *Int. J. Secur. Netw.* 8, 4 (2013), 179–193.
- Davide Zordan, Borja Martinez, Ignasi Vilajosana, and Michele Rossi. 2012. To compress or not to compress: Processing vs transmission tradeoffs for energy constrained sensor networking. *CoRR*, arXiv preprint, abs/1206.2129:1206.2129 (2012).
- Davide Zordan, Tommaso Melodia, and Michele Rossi. 2016. On the design of temporal compression strategies for energy harvesting sensor networks. *IEEE Trans. Wireless Commun.* 15, 2 (2016), 1336–1352.

Received November 2016; revised April 2017; accepted July 2017